

Neural networks - Project 3

Facial Recognition using Siamese Neural Networks

Lenka Husárová, Martin Mislovič, Hana Magyarová

Last updated: May 14, 2025

✓ Task

Our task was to develop and evaluate a face verification system. The approach involved training Siamese networks with triplet loss to learn facial embeddings that distinguish between different identities. Specifically, the task included comparing Siamese networks trained with classical triplet loss and shadow loss. Additionally, the project aimed to explore the effects of different model hyperparameters and embedding sizes. The system involved dataset preparation, triplet sampling, model design, training, and performance evaluation using verification metrics the system used the metrics mentioned below with the goal of accurately verifying whether two facial images belong to the same person.

PAPER: <https://arxiv.org/pdf/2311.14012>

✓ Metrics

We used a Siamese Network for face recognition and evaluated its performance using the following metrics:

- TAR@FAR=10%
- TAR@FAR=5%
- TAR@FAR=1%
- AUC (Area Under Curve)
- EER (Equal Error Rate)

- EER Threshold

TAR (True Accept Rate) at a given FAR (False Accept Rate) helps us understand how well the model verifies identities while controlling for a specific false acceptance level. **AUC** gives an overall view of the model's ability to separate positive and negative pairs. **EER** indicates the point where false accept and false reject rates are equal, giving a balanced performance score. We also used **ROC curves** to visualize model performance and tracked the metrics during training and validation using Weights & Biases.

✓ Code

- **EDA.ipynb** - This notebook contains our initial data exploration

Key activities include:

- analyzing the distribution of images across different classes
- examining image resolutions and size variance
- identifying class imbalance
- determining the quality of the images

- **model.ipynb** - This notebook focuses on preprocessing, building, training, and evaluating models.

It includes:

- hyperparameter sweeps using Bayesian optimization
- comparison between Shadow Loss and Triplet Loss
- experiments on training with only valid classes
- evaluation of a simplified ResNet18-based architecture

- training and testing of the best-performing model configuration

✓ Dataset

Link: <https://www.kaggle.com/datasets/atulanandjha/lfwpeople>

This dataset contains more than 13,000 images of faces collected from the web. Each face has been labeled with the name of the person pictured. 1680 of the people pictured have two or more distinct photos in the data set. The only constraint on these faces is that they were detected by the Viola-Jones face detector.

✓ Preprocessing

We began with dataset preprocessing, where we used the ImageFolder format and implemented a custom function to split the dataset into training, validation, and testing subsets. Each subset was formed by ensuring identities (classes) were exclusive to only one of the splits, with **80%** of identities used for **training**, and **10%** each for **validation** and **testing**. To facilitate training with triplet loss, we designed an **OnlineTripletDataset** class that dynamically samples triplets—**anchor**, **positive**, and **negative** images—based on class labels, ensuring that each triplet meets the requirements of similarity and dissimilarity in embedding space. Additionally, a separate function was implemented to create labeled **image pairs** (same or different identities) from the test set, which is necessary for the final verification stage.

Models and training



We implemented two convolutional neural network architectures.

✓ ResNet18

The first model is a straight-up version of ResNet18. It uses a bunch of stacked residual blocks that help the network learn more effectively and tackle issues like vanishing gradients. Each of these blocks has two convolutional layers, with batch normalization and ReLU activation. The model gradually shrinks the spatial dimensions while deepening the feature maps, ultimately leading to a global average pooling layer and a fully connected layer at the end.

✓ Simplified ResNet18

The second model is a simpler take on ResNet18. It has fewer residual blocks and less channel expansion, making it a lighter model. Even with its reduced depth, it sticks to the essential structure of convolutional layers, normalization, and activation layers, finishing off with adaptive average pooling and a linear projection to the embedding space.

✓ Experiments

To evaluate and improve our deep metric learning approach, we conducted a series of targeted experiments:

- **Shadow Loss vs. Triplet Loss:** We compared two loss functions—Shadow Loss and the standard Triplet Loss—to assess their effectiveness in learning discriminative embeddings.
- **Only Valid Classes:** We compared the effects of class selection on the dataset.

- **Simplified ResNet18:** We tested a lightweight, simplified version of ResNet18 to explore whether a reduced model architecture could maintain performance while lowering computational costs.
- **Best Model Evaluation:** Finally, we trained and evaluated our best-performing model configuration based on insights from the previous experiments, using optimized hyperparameters and full dataset.

For our experiments we used the following sweep config to find the best possible parameters each time:

```
sweep_config = {
    "method": "bayes",
    "metric": {
        "name": "validation_eer",
        "goal": "minimize"
    },
    "parameters": {
        "batch_size": {
            "min": 16,
            "max": 64
        },
        "margin": {
            "min": 0.0,
            "max": 1.0,
        },
        "lr": {
            "min": 0.00001,
            "max": 0.001
        },
        "weight_decay": {
            "min": 0.01,
            "max": 1.0
        },
        "embedding_size": {
            "min": 32,
            "max": 256
        }
    }
}
```

All the experiments shared the same basic setup:

- AdamW optimizer
- CosineAnnealing learning rate scheduler with `eta_min=1e-6` and `T_max=10`
- 10 Epochs

✓ Shadow loss vs. Triplet loss

In this experiment, we aim to compare the performance of Shadow Loss and Triplet Margin Loss. The Shadow Loss Sweep involved training the model with Shadow Loss and the Triplet Loss Sweep used the traditional Triplet Margin Loss. Both sweeps used the same ResNet18 architecture and were trained for 10 epochs and their performance was evaluated using the test subset and test pairs.

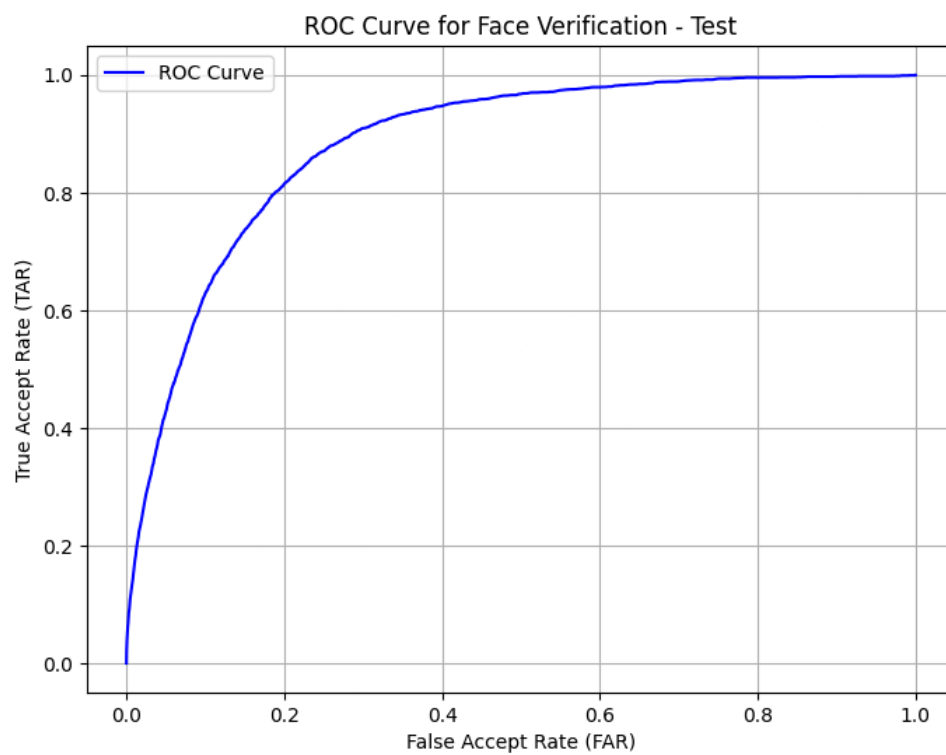
✓ Results for shadow loss

The best run had the following parameters:

```
batch_size: 58
embedding_size: 124
lr: 0.0008053248712495344
margin: 0.10797025170296592
weight_decay: 0.4707712240478096
```

Results:

```
--- Test ---
TAR@FAR=10%: 0.6297872340436784
TAR@FAR=5%: 0.4288753799409456
TAR@FAR=1%: 0.16717325228216662
AUC: 0.8840634506382051
EER: 0.19386759289334382
EER Threshold: 1.1474634150529768
```

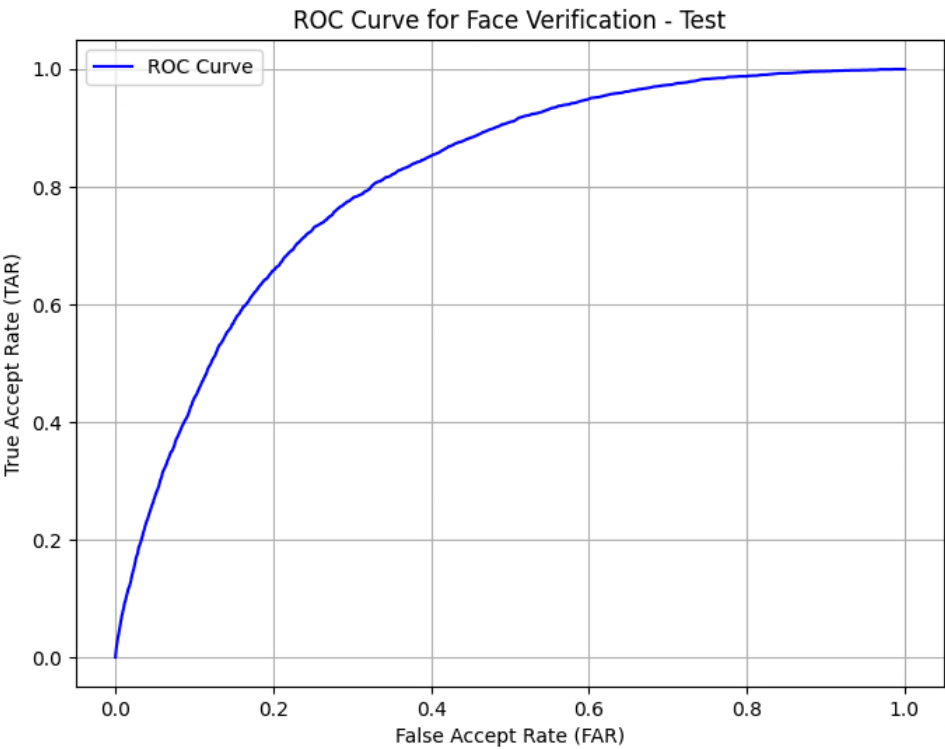
✓ Results for triplet loss

The best run had the following parameters:

```
batch_size: 53  
embedding_size: 102  
lr: 0.0007585815420565159  
margin: 0.7309087703710409  
weight_decay: 0.7667778227130992
```

Results:

```
--- Test ---  
TAR@FAR=10%: 0.4413373860199351  
TAR@FAR=5%: 0.2738601823730278  
TAR@FAR=1%: 0.08054711246480073  
AUC: 0.811372488247512  
EER: 0.2616239945419902  
EER Threshold: 0.8288578130043781
```



✓ Comparison of results:

We evaluated the performance of our ResNet18-based model using two different loss functions: Shadow Loss and Triplet Loss. The comparison is based on standard biometric verification metrics:

Metrics	Shadow Loss	Triplet Loss
TAR @ FAR = 10%	0.6298	0.4413
TAR @ FAR = 5%	0.4289	0.2739
TAR @ FAR = 1%	0.1672	0.0805
AUC	0.8841	0.8114
EER	0.1939	0.2616
EER Threshold	1.1475	0.8289

Shadow Loss consistently **outperforms Triplet Loss** across all evaluation metrics. It yields significantly higher TAR at all FAR levels, achieves a better AUC, and results in a lower EER. These

results suggest that Shadow Loss is more effective for learning embeddings in our architecture.

There is a possibility that triplet loss could achieve better results with a combination of parameters undiscovered by the sweep or with different data split. Across our testing we have however never encountered a case where Triplet Loss consistently outperformed Shadow Loss.

✓ Only valid classes

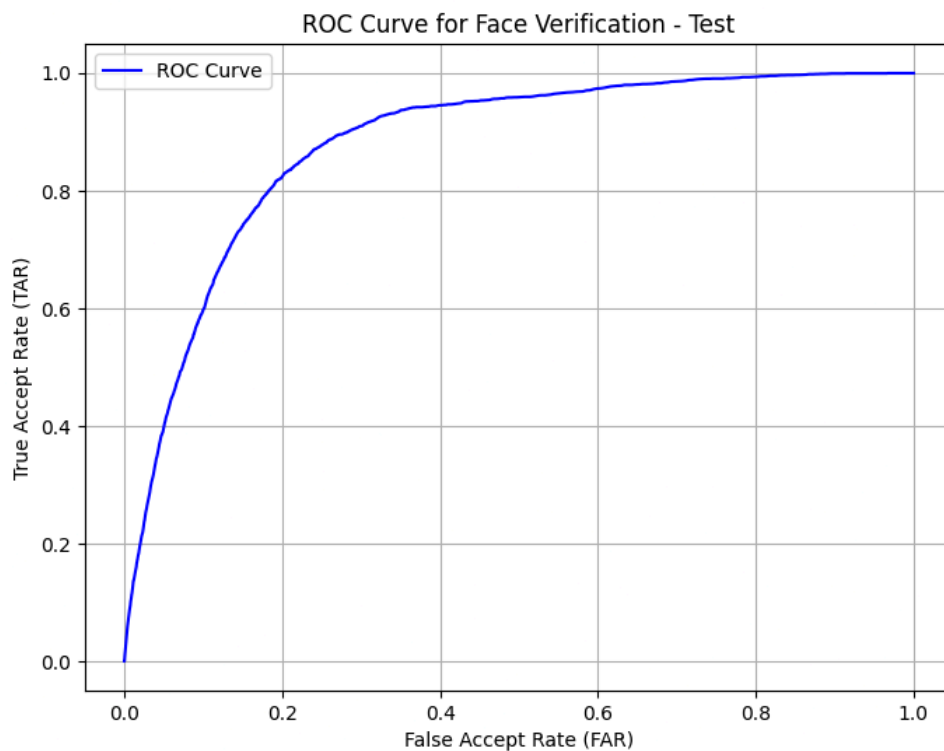
In this experiment we wanted to see how would the overall performance be impacted if we selected only classes that contain two or more images (this practically only influences which negatives are available for training). We took the same configuration as the Shadow Loss test above and tried it with this more constrained class selection.

The best run had the following parameters:

```
batch_size: 50  
embedding_size: 232  
lr: 0.0007278967027688243  
margin: 0.7436453775835431  
weight_decay: 0.6581442379140964
```

Results:

```
--- Test ---  
TAR@FAR=10%: 0.5981828839402163  
TAR@FAR=5%: 0.39917936694197187  
TAR@FAR=1%: 0.11987104337889831  
AUC: 0.8792620901869042  
EER: 0.18940977312202098  
EER Threshold: 0.9669229924544558
```



We can see that the model performed fairly well giving us a slightly better EER than before. However the TAR@FAR=10% and AUC if this model were worse. Its also worth mentioning that since the class

selection is more limited, test dataset is smaller for this experiment. In the end we decided that improvements this approach brought didn't justify the potential downsides.

✓ Simplified ResNet18

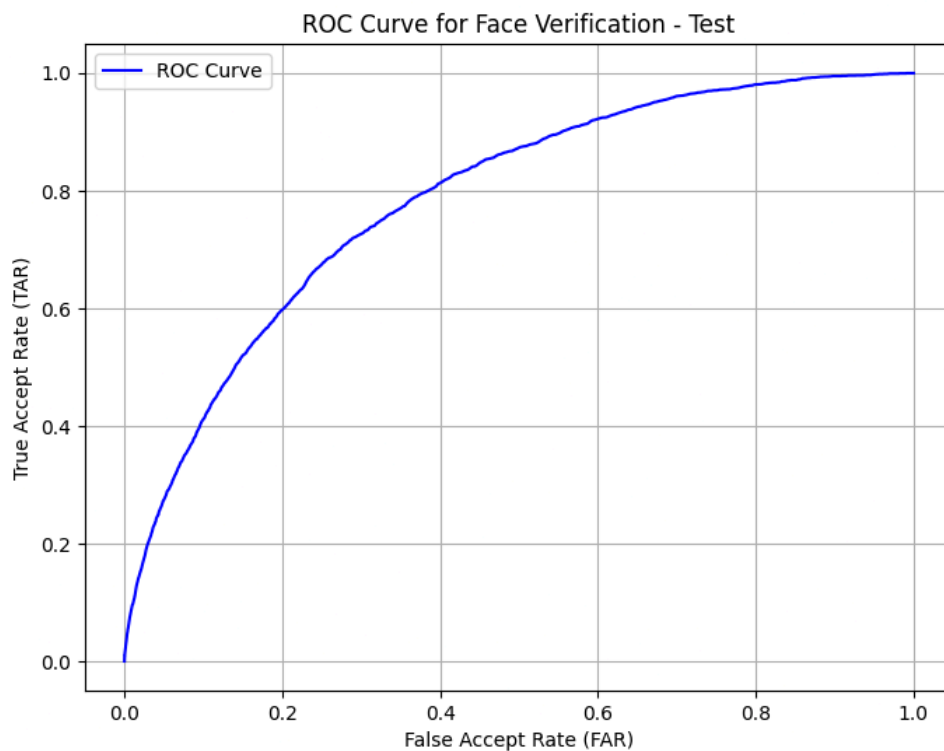
The SimplifiedNeuralNet model, based on a simplified ResNet-18 architecture, was trained using the Shadow Loss to learn. A hyperparameter sweep was conducted to explore variations in batch size, embedding size, margin, learning rate, and weight decay in order to identify the optimal configuration. The training used the AdamW optimizer with a CosineAnnealingLR scheduler to adjust the learning rate dynamically over 10 training epochs.

The best run had the following parameters:

```
batch_size: 53
embedding_size: 164
lr: 0.00010434959467286711
margin: 0.602501015647548
weight_decay: 0.9492008506893712
```

Results:

```
--- Test ---
TAR@FAR=10%: 0.41155015197747247
TAR@FAR=5%: 0.27537993921192894
TAR@FAR=1%: 0.09604863222159254
AUC: 0.7851280255418426
EER: 0.28500948567196394
EER Threshold: 1.0590938469907782
```



In the end the simpler model performed worse than the full resnet18.

✓ Best model

In our experiment, we employed the NeuralNet (ResNet18) model with an embedding size of 170, in combination with the ShadowLoss function using a margin of 0.25, to evaluate its performance in a similarity detection task. The model was trained using the AdamW optimizer with a learning rate of 0.0002 and a weight decay of 0.75. To adjust the learning rate over the training process, we used a CosineAnnealingLR scheduler with a maximum of 15 iterations and a minimum learning rate of 1e-6. Training was conducted for 15 epochs with a batch size of 64 on both the training and validation datasets.

The results of this experiment:

Parameters:

```
batch_size: 64  
embedding_size: 170  
lr: 0.0002  
margin: 0.25  
weight_decay: 0.75
```

Results:

--- Test ---

TAR@FAR=10%: 0.7021369385091536

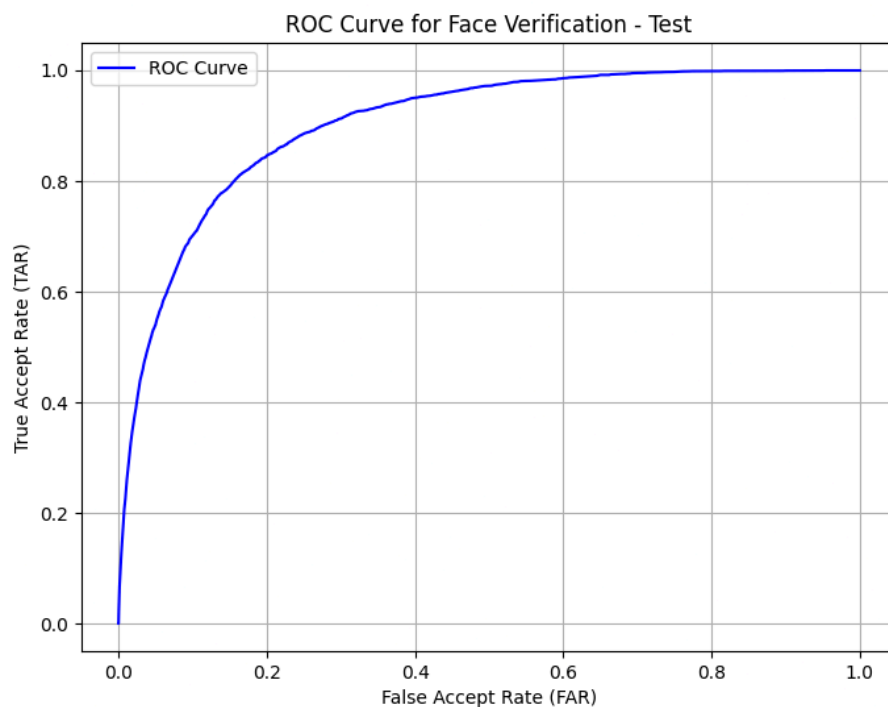
TAR@FAR=5%: 0.5390318360236829

TAR@FAR=1%: 0.2396423898839083

AUC: 0.9031835135218993

EER: 0.17708437348664405

EER Threshold: 0.9779676892258622



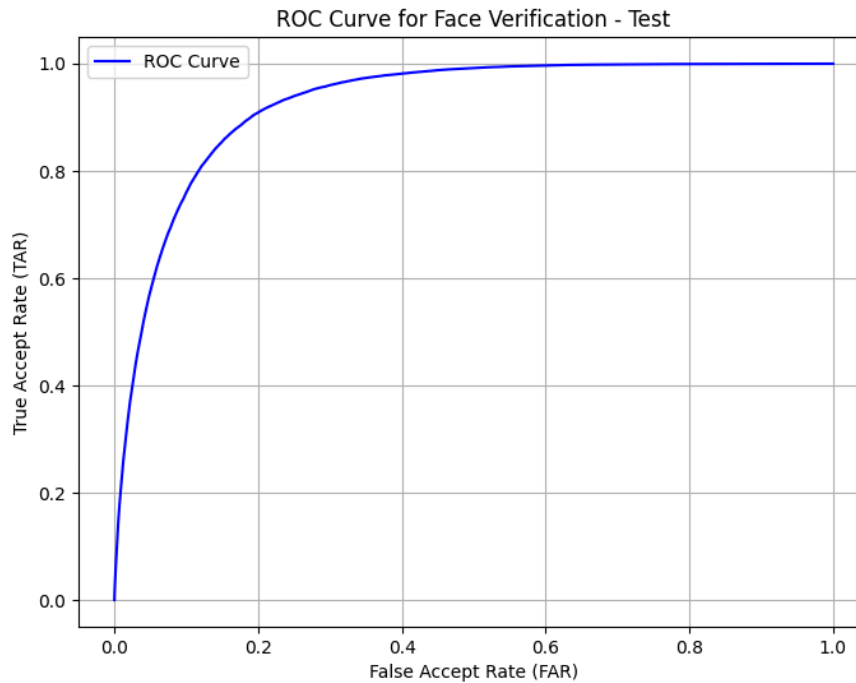
Our best results so far from this experiment. We achieved an EER of 0.148.

Parameters:

```
batch_size: 64
embedding_size: 170
lr: 0.0002
margin: 0.3
weight_decay: 0.75
```

Results:

```
--- Test ---
TAR@FAR=10%: 0.7601799775028892
TAR@FAR=5%: 0.5770850072313749
TAR@FAR=1%: 0.22262574321092
AUC: 0.9262120610420923
EER: 0.1480814251437618
EER Threshold: 0.9036553815320447
```



Both runs were conducted with almost the same parameter settings, yet provided slightly different results. We attribute this difference to different split of classes across the dataset (more informative classes ended up in train). There is a bit of randomness involved when selecting the positive and negative images but we don't believe this randomness has significant influence on the training process as statistically speaking model should see all the images several times during training.

✓ Comparison with paper

In our project, we aim to investigate the effectiveness of Shadow Loss as an alternative to the traditional Triplet Loss function in Siamese networks. According to findings from paper, Shadow Loss offers several advantages, particularly in terms of memory efficiency and computational speed, while maintaining or even improving performance. The core idea of Shadow Loss lies in its ability to project embeddings onto a lower-dimensional space, which reduces memory usage and accelerates convergence during training.

Our experimental results support these claims. When applied to the same ResNet18-based architecture, Shadow Loss outperformed Triplet Loss across all key evaluation metrics. Specifically, it achieved higher TAR at every FAR level, a better AUC (0.884 vs. 0.811), and a lower EER (0.1939 vs. 0.2616). These improvements demonstrate that Shadow Loss is better in learning discriminative embeddings for verification tasks in our setup.

We however weren't able to observe any significant improvements in training time but it is possible that our dataset simply wasn't large enough for these differences to be observable.

Overall the Shadow Loss functions seemed to converge faster than standard Triplet Loss. Indirect consequence of this in our experimentation was that it was more prone to overfitting with incorrect hyperparameters.

✓ Summary

Our primary objective was to evaluate Shadow Loss compared to the traditional Triplet Loss function. Through extensive experiments, including hyperparameter optimization, architecture simplification, and dataset refinement, we demonstrated that Shadow Loss improves verification performance. Key results showed that models trained with Shadow Loss achieved higher TAR, lower EER, and better AUC scores than those trained with Triplet Loss. In the end, we ran the model that achieved the best results.

<https://wandb.ai/xmagyarova-stu/nsiete-project-3/reports/Neural-networks-Project-3--VmllldzoxMjczOTkwMg>